

# Unit 2

# Machine Learning Preliminaries

**Dr. A N Ramya Shree**  
**Dept. of CSE(AI&ML)**

# Machine Learning Preliminaries

- Hold out validation to assess model performance.
- K-fold cross-validation to assess model performance.
- Handling categorical data.
- Feature selection techniques.
- Evaluation Metrics
  - Classifier evaluation
  - Regression evaluation

# Variance

- Variance is a statistical measure that indicates how much individual values in a dataset differ from the mean (average) of the dataset.

$$\text{Variance} = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$$

- A high variance indicates that the data points are spread out from the mean, suggesting that the values are widely dispersed.
- A low variance indicates that the data points are close to the mean, suggesting that the values are more tightly grouped.

# Bias

- Bias is the error introduced by approximating a real-world problem (which might be complex) with a simplified model.
- Bias is the error due to the model's simplistic assumptions about the data.
- It represents the model's inability to capture the underlying patterns in the data.

**A model with high bias pays too little attention to the training data - when the model is too simplistic and cannot capture the relationships between input features and target outputs accurately.**

# bias-variance tradeoff

- The bias-variance tradeoff is about finding a sweet spot between these two errors:
- **High Bias, Low Variance:** A model that is too simple (high bias) doesn't capture the complexity of the data and is consistent across different training sets( underfits).
- **Low Bias, High Variance:** A model that is too complex (low bias) captures the training data very well but is overly sensitive to its specific details, leading to poor generalization on new data (high variance).
- **Ideal Balance:** The goal is to find a model that has low enough bias to capture the main patterns in the data, but not so much flexibility that it leads to high variance. This model will generalize well to new data.

- **Algorithmic Bias:** Bias that arises from the choice of model or algorithm, such as assuming linear relationships in a dataset with nonlinear patterns.
- **Sampling Bias:** Bias introduced when the training data does not represent the population well, leading the model to learn patterns that may not apply to the real-world data it encounters later.
- **Measurement Bias:** Bias from inaccurate data measurements, which can cause the model to learn incorrect patterns or relationships.

# Underfit and Overfit in ML

- In machine learning, underfitting and overfitting are two common problems related to model performance and generalization.

## **Underfitting**

- Underfitting occurs when a model is too simple to capture the underlying patterns in the data.
- This means the model performs poorly on both the training data and new, unseen data.

## **Causes**

- A model that's too simple for the data.
- Insufficient training, meaning the model hasn't learned enough from the data.
- Inadequate features, where the available data doesn't contain enough relevant information for learning

## **Symptoms:**

High error on both the training set and the test set.

# How to overcome?

- Choose a more complex model
- Train the model for more epochs or iterations.
- Add more relevant features to better capture the underlying patterns in the data.



## **Overfitting:**

Overfitting happens when a model learns the training data too well, including noise and outliers, making it too specialized.

This results in excellent performance on the training data but poor generalization to new data.

### **Causes:**

A model that's too complex for the data, capturing noise as if it were true patterns.

Excessive training, which leads the model to memorize rather than learn general trends.

### **Symptoms:**

Very low error on the training set, but high error on the test set.

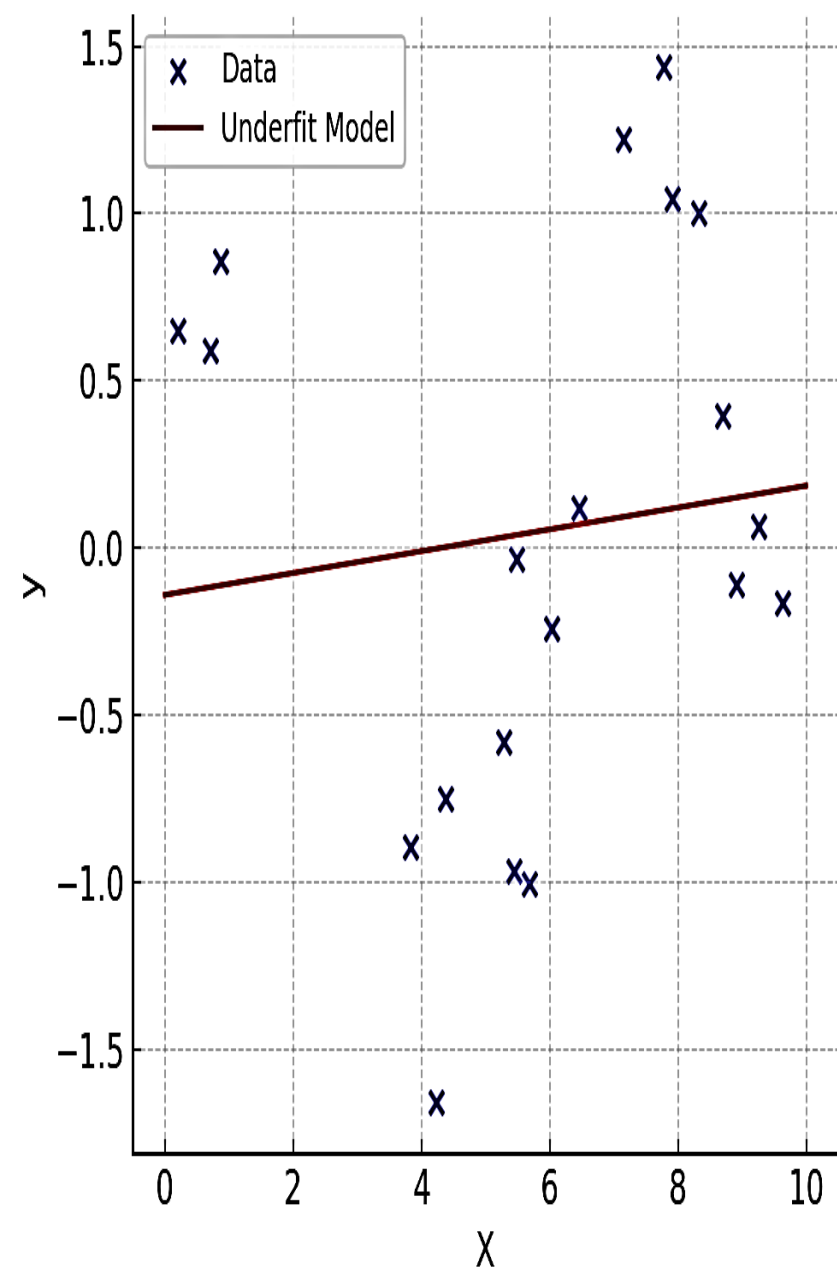
# How to overcome?

- Use k-fold technique
- Use regularization
- Use more training data, which can help the model better generalize

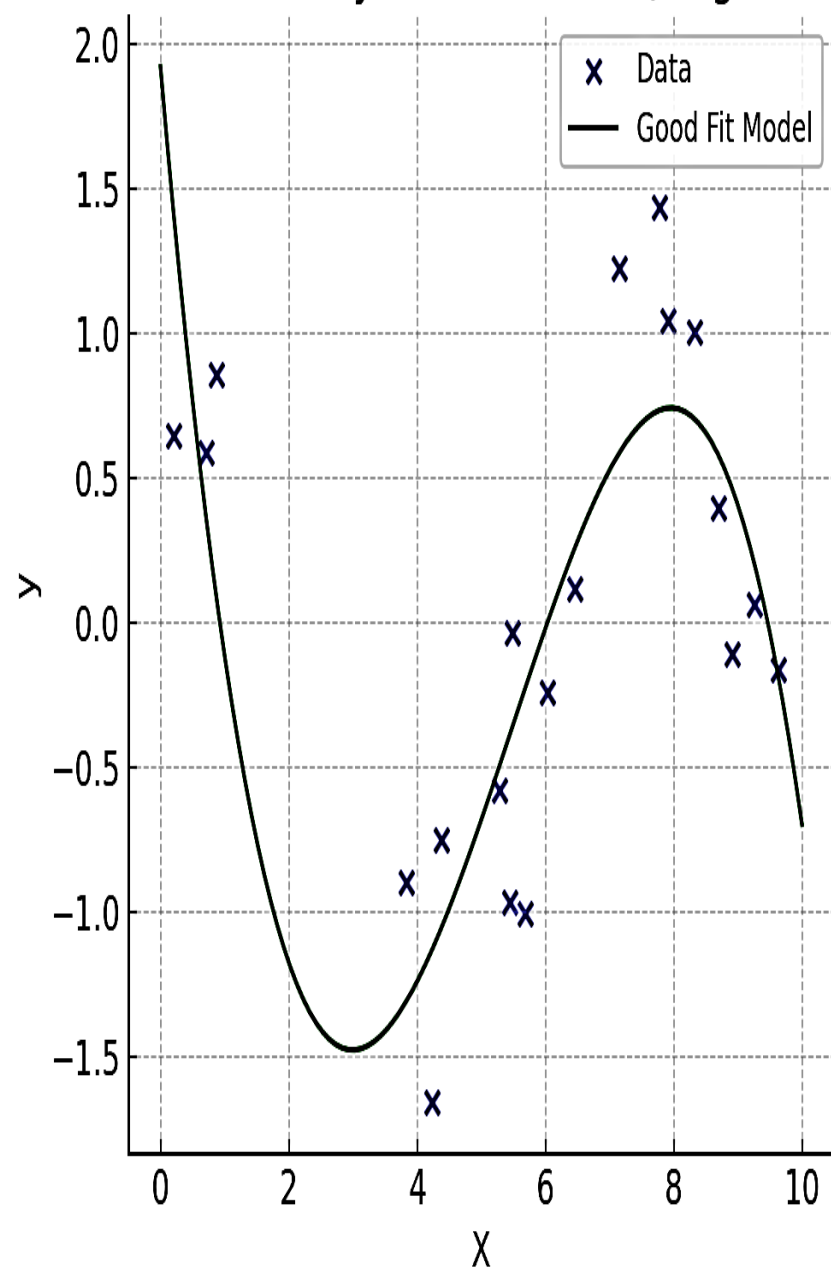
# If we imagine fitting a curve to some data

- **Underfit:** The curve might be a simple straight line that doesn't capture the pattern.
- **Overfit:** The curve might be wavy, fitting through all points, even noise.
- **Good Fit:** The curve captures the general trend without following every data point too closely.

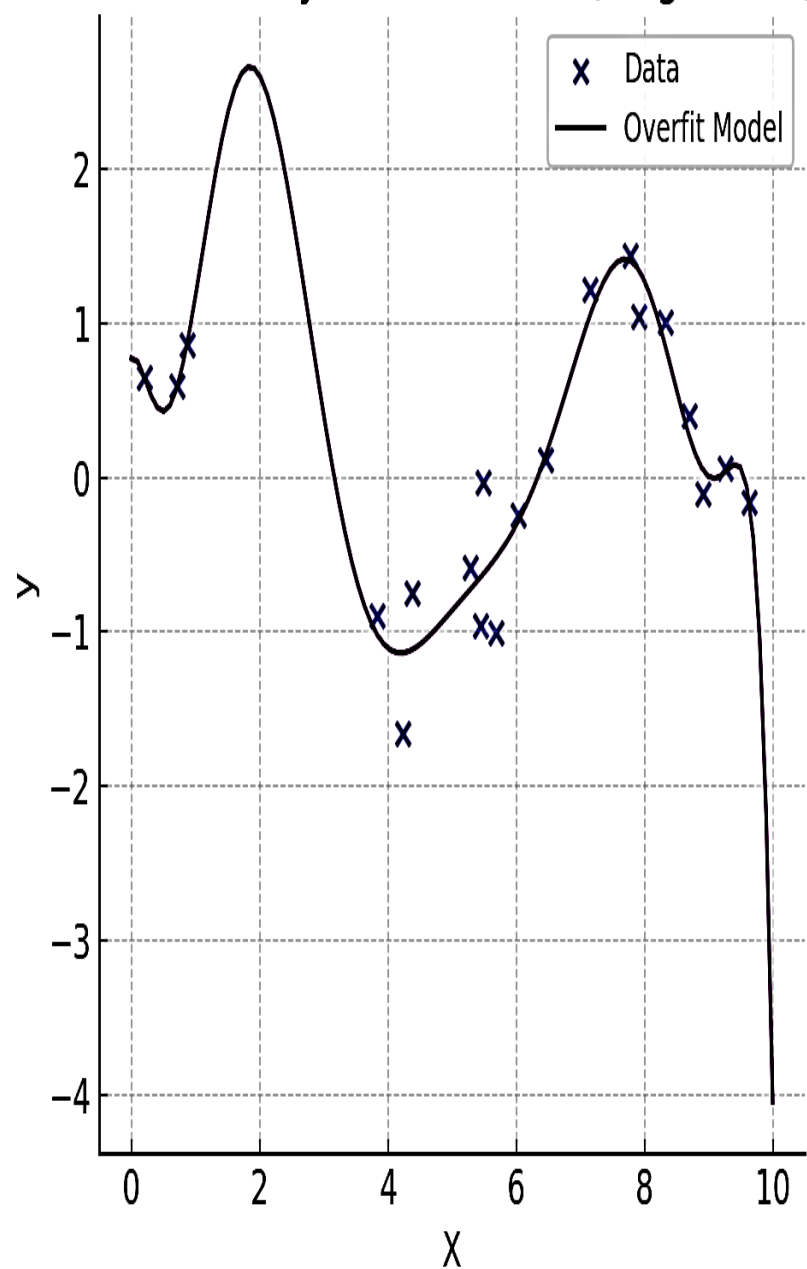
### Underfit: Linear Model

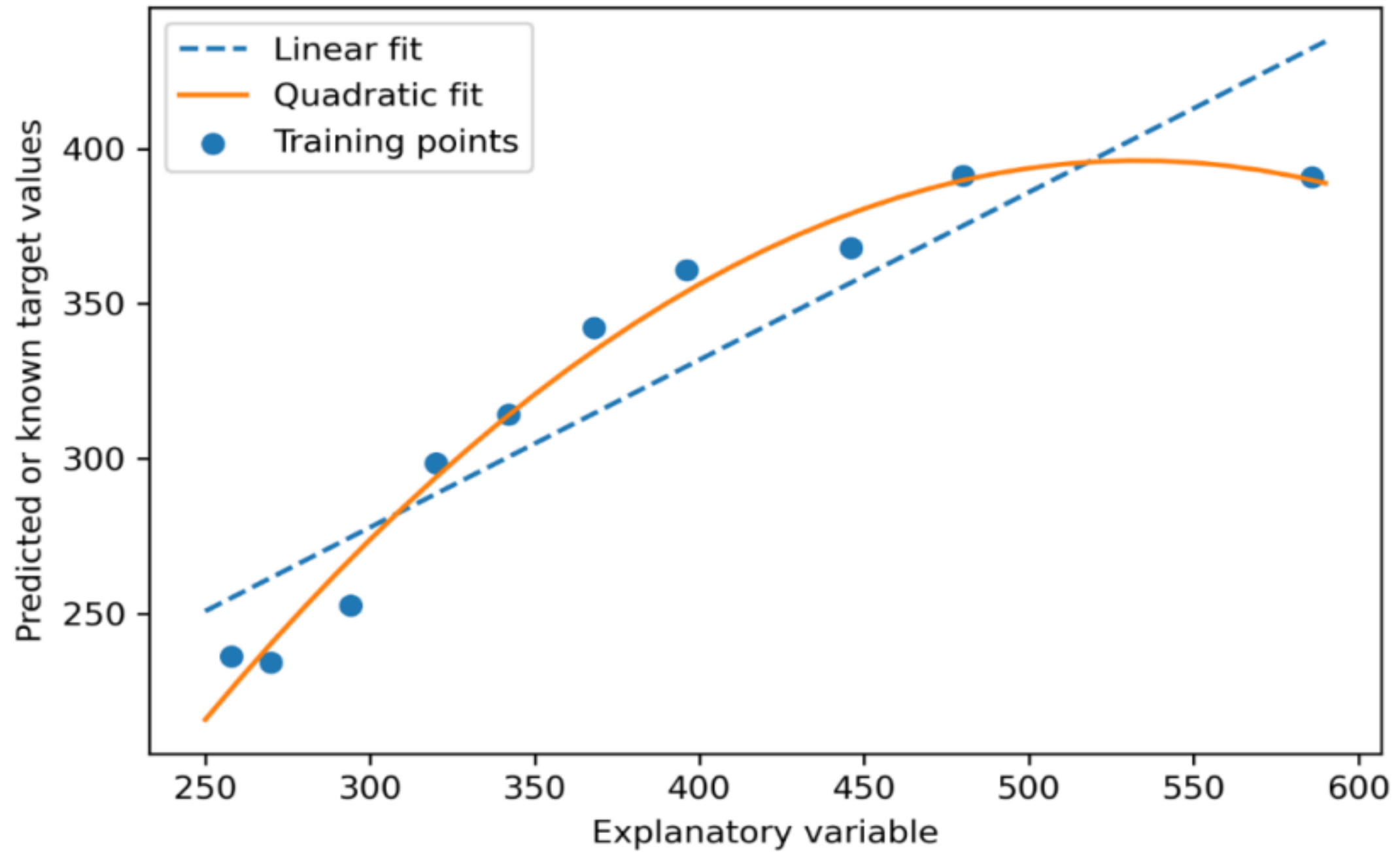


### Good Fit: Polynomial Model (Degree 3)



### Overfit: Polynomial Model (Degree 10)

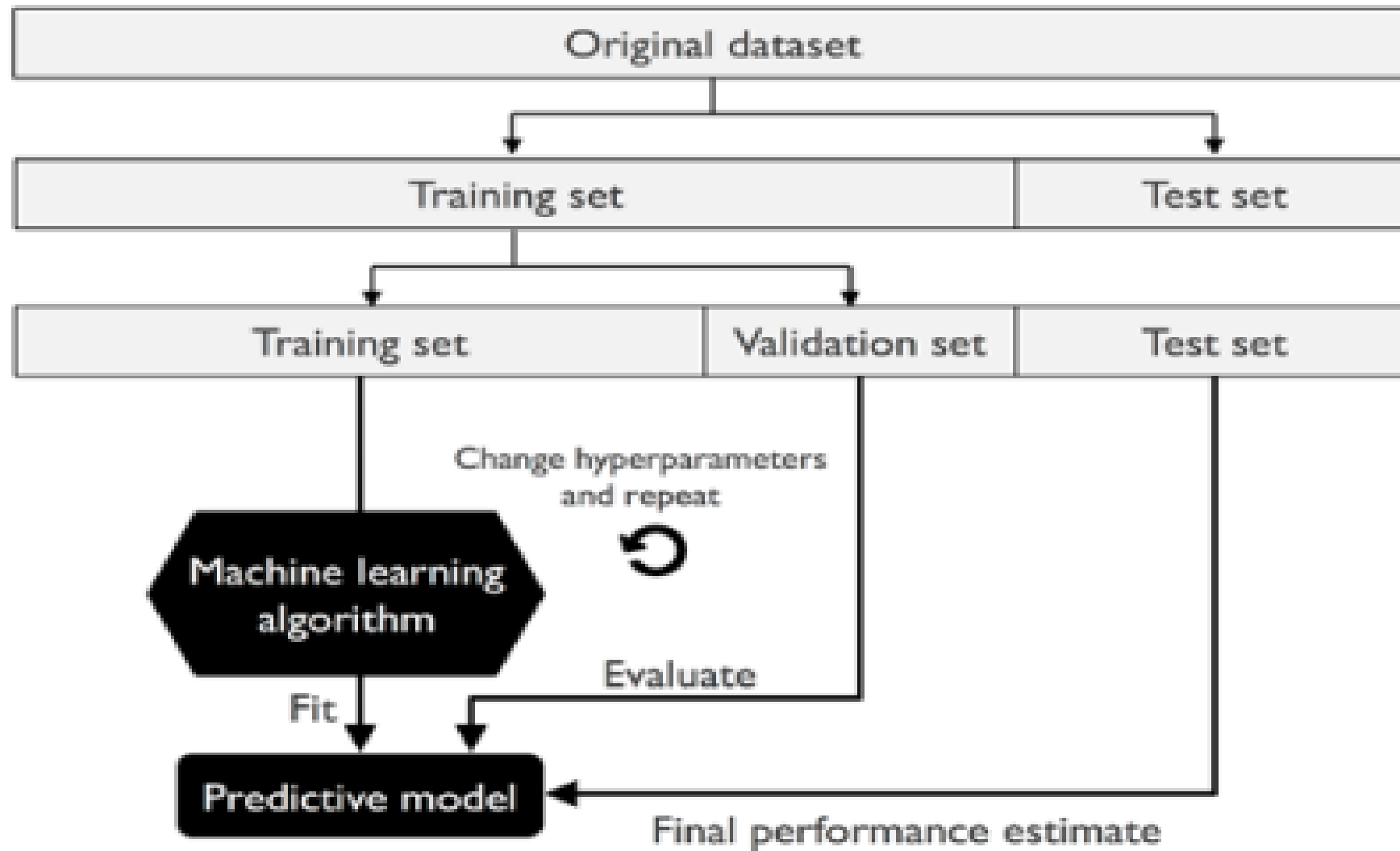




# Holdout validation to assess model performance.

- The holdout method is a classic and popular approach for estimating the generalization performance of machine learning models.
- Using the holdout method, we split our initial dataset into separate training and test datasets.
- The former is used for model training, and the latter is used to estimate its generalization performance.
- A disadvantage of the holdout method is that the performance estimate may be very sensitive to how
- We partition the training dataset into the training and validation subsets the estimate will vary for different examples of the data.

- A disadvantage of the holdout method is that the performance estimate may be very sensitive to how we partition the training dataset into the training and validation subsets.
- The estimate will vary for different examples of the data.





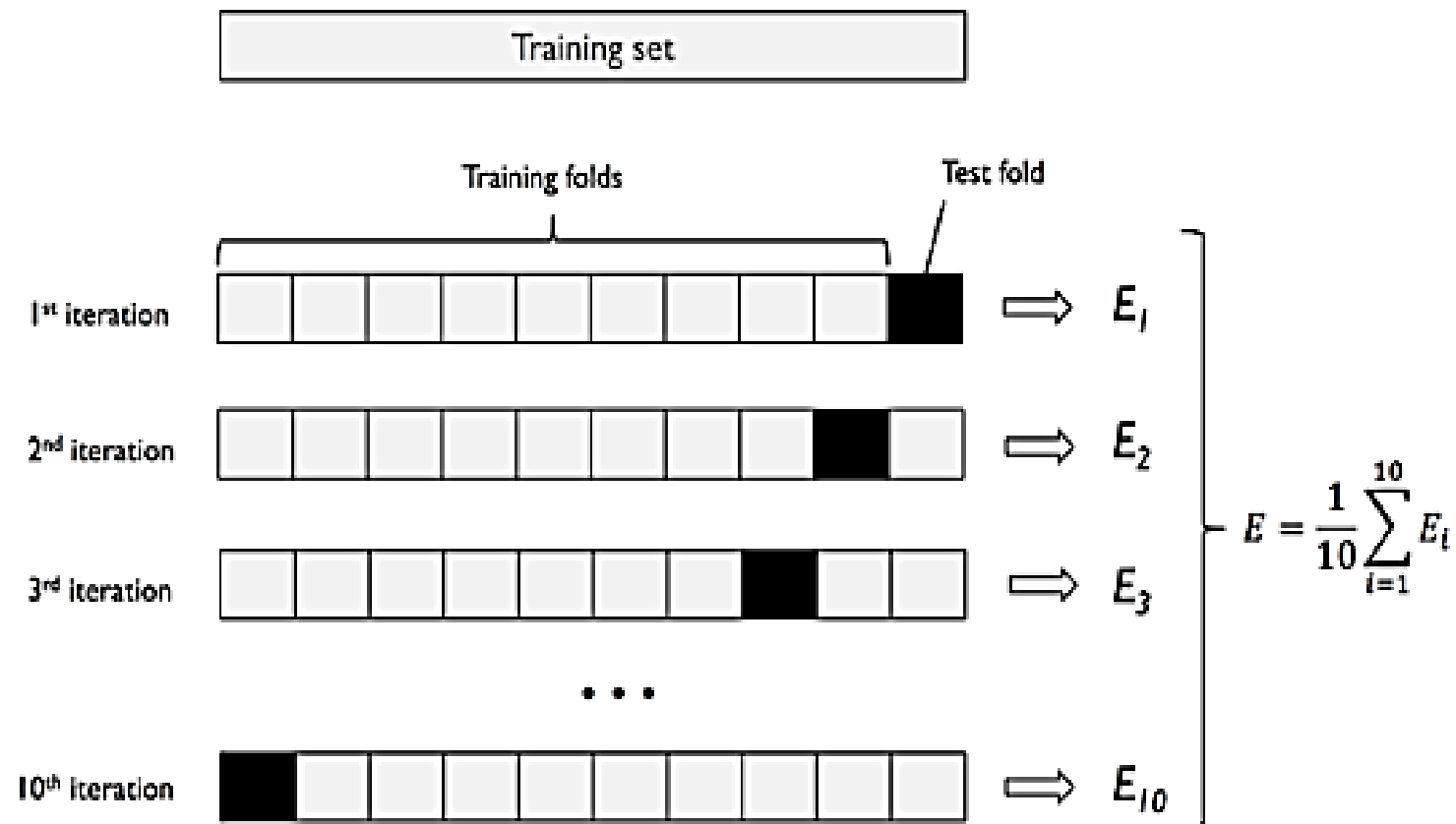
# k-fold cross-validation to assess model performance

- k-fold cross-validation, where we repeat the holdout method  $k$  times on  $k$  subsets of the training data.
- In k-fold cross-validation, we randomly split the training dataset into  $k$  folds without replacement.
- Here,  $k - 1$  folds, the so-called training folds, are used for the model training, and one-fold, the so-called test fold, is used for performance evaluation.
- This procedure is repeated  $k$  times so that we obtain  $k$  models and performance estimates.

- The rationale behind fitting a model to the whole training dataset after k-fold cross-validation is that :
- first, we are typically interested in a single, final model (versus k individual models).
- second, providing more training examples to a learning algorithm usually results in a more accurate and robust model.

- Since k-fold cross-validation is a resampling technique without replacement, the advantage of this approach is that in each iteration, each example will be used exactly once, and the training and test folds are disjoint.
- k-fold cross-validation with  $k = 10$ . The training dataset is divided into 10 folds, and during the 10 iterations, 9 folds are used for training, and 1 fold will be used as the test dataset for model evaluation.
- In summary, k-fold cross-validation makes better use of the dataset than the holdout method with a validation set, since in k-fold cross-validation all data points are being used for evaluation.

Estimated performances,  $E_i$  for each fold are then used to calculate the estimated average performance,  $E$ , of the model.



# Advantages

- **Reduces Overfitting:** By training on different portions of the dataset, K-fold cross-validation helps assess the model's performance on different data distributions.
- **Improves Generalization:** The average performance over K iterations is generally a better estimate of how well the model will perform on new data.
- **Efficient Use of Data:** Unlike traditional train-test splits where a single portion is left aside for testing, K-fold cross-validation makes efficient use of all available data, as every observation is used for both training and validation.

- However, if we are working with relatively small training sets, it can be useful to increase the number of folds.
- If we increase the value of  $k$ , more training data will be used in each iteration, which results in a lower pessimistic bias toward estimating the generalization performance by averaging the individual model estimates.
- However, large values of  $k$  will also increase the runtime of the cross-validation algorithm and yield estimates with higher variance, since the training folds will be more like each other.
- On the other hand, if we are working with large datasets, we can choose a smaller value for  $k$ , for example,  $k = 5$ , and still obtain an accurate estimate of the average performance of the model while reducing the computational cost of refitting and evaluating the model on the different folds.

- A slight improvement over the standard k-fold cross-validation approach is stratified k-fold cross-validation, which can yield better bias and variance estimates, especially in cases of unequal class proportions.
- In stratified cross-validation, the class label proportions are preserved in each fold to ensure that each fold is representative of the class proportions in the training dataset.

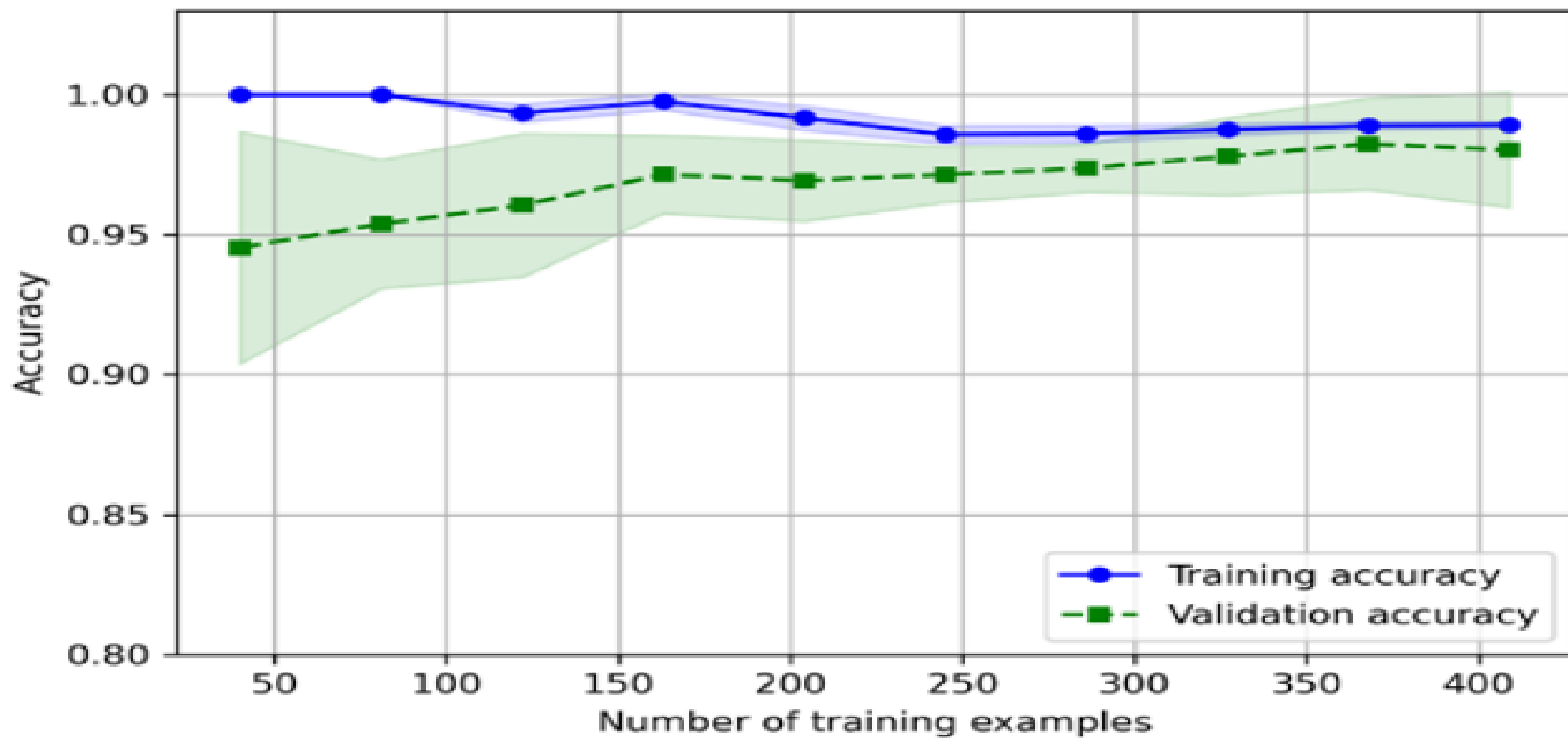
```
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier
import numpy as np

data = load_iris()
X = data.data
y = data.target
model = DecisionTreeClassifier()
kf = KFold(n_splits=10, shuffle=True, random_state=1)
cv_scores = cross_val_score(model, X, y, cv=kf)
print("Scores for each fold:", cv_scores)
print("Average Score:", np.mean(cv_scores))
print("Standard Deviation:", np.std(cv_scores))
```



- Scores for each fold: [0.93 0.87 0.93 1. 0.93 0.93 0.93 0.87 1. 0.93]
- Average Score: 0.93
- Standard Deviation: 0.045

A learning curve showing training and validation dataset accuracy by the number of training examples



# Class Imbalance

- Class imbalance is a common challenge in machine learning, especially in classification tasks where one class has significantly fewer instances than others.
- This imbalance can lead to biased models that perform poorly on the minority class.

# Challenges of Class Imbalance

- **Biased Model Predictions:** A model trained on imbalanced data may become biased toward predicting the majority class.
- **Metric Misinterpretation:** Metrics like accuracy may appear high, but they fail to reflect the model's true performance on the minority class.
- **Poor Generalization:** The model may not generalize well to new data, especially for minority class instances, leading to higher misclassification rates.

# How to deal with Class imbalance?

## 1. Resampling Techniques

- **Oversampling:** Increase the number of samples in the minority class by duplicating or generating synthetic data.
- Common methods include:
  - Random Oversampling: Simply duplicating minority class samples.
  - SMOTE (Synthetic Minority Over-sampling Technique):  
Generates synthetic samples by interpolating between existing ones in the minority class.
- **Under sampling:** Reduce the number of samples in the majority class by randomly removing some of them to balance the dataset.
- **Random Under sampling:** Randomly drops samples from the majority class.
- **NearMiss:** Selects the majority samples that are closest to the minority samples based on distance metrics.
- **Hybrid Sampling:** Combines both oversampling and undersampling for a more balanced approach.

## 2. Use of Class-Weighted Algorithms

- Many machine learning algorithms allow you to assign different weights to classes.
- By assigning a higher weight to the minority class, the model learns to give more importance to it.

### 3. Use of Different Evaluation Metrics

- When dealing with imbalanced data, accuracy may not be the best metric.
- Instead, use metrics that focus on the minority class, Precision, Recall, and F1-Score.
- Especially useful in binary classification to evaluate the performance on minority class.
- **ROC-AUC:** Measures the model's ability to distinguish between classes.
- **PR-AUC (Precision-Recall AUC)** is often more informative when dealing with high class imbalance.

# ML Classification Model Evaluation

- When evaluating machine learning (ML) models, selecting the right metrics is crucial, as it impacts model performance assessment and improvement.
- The appropriate metric depends on the type of problem, such as classification, regression, or clustering.
- A confusion matrix, a matrix that lays out the performance of a learning algorithm.
- A confusion matrix is simply a square matrix that reports the counts of the true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions of a classifier.



# Confusion Matrix

|              |   | Predicted class      |                      |
|--------------|---|----------------------|----------------------|
|              |   | P                    | N                    |
| Actual class | P | True positives (TP)  | False negatives (FN) |
|              | N | False positives (FP) | True negatives (TN)  |

- True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) are terms used to describe the outcomes of predictions.
- These are derived from a confusion matrix, which helps evaluate the performance of a model by categorizing predictions based on their correctness and the actual class.

# True Positive (TP)

A True Positive occurs when the model correctly predicts the positive class.

In a medical test for a disease, a TP would be a correct prediction where the model identifies a patient as having the disease, and they actually do.

# True Negative (TN)

A **True Negative** occurs when the model correctly predicts the negative class.

Using the medical test scenario, a TN would be a correct prediction where the model identifies a patient as not having the disease, and they actually don't.

# False Positive (FP) (Type I Error)

A False Positive occurs when the model incorrectly predicts the positive class.

In the medical test scenario, an FP would be when the model identifies a patient as having the disease when they actually don't.

# False Negative (FN) (Type II Error)

A False Negative occurs when the model incorrectly predicts the negative class.

In the medical test scenario, a FN would be when the model predicts a patient as not having the disease when they actually do.

- TP and TN indicate correctly classified instances, representing the model's ability to predict accurately.
- FP and FN indicate misclassified instances, showing where the model's predictions are incorrect.
- These values are used to calculate performance metrics like accuracy, precision, recall, and F1 score, which provide deeper insights into a model's performance, especially for imbalanced datasets.

# Accuracy

- Measures the ratio of correctly predicted instances to the total instances.
- $\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$
- Useful for balanced datasets but less informative for imbalanced classes.
- Class imbalance occurs when one class in a dataset significantly outnumbers the other(s), which can lead to issues with model training and evaluation.
- In such cases, standard metrics like accuracy may become misleading, as the model can achieve high accuracy by simply predicting the majority class all the time without learning useful patterns.



$$ERR = \frac{FP + FN}{FP + FN + TP + TN}$$

The prediction accuracy can then be calculated directly from the error:

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} = 1 - ERR$$

The **true positive rate (TPR)** and **false positive rate (FPR)** are performance metrics that are especially useful for imbalanced class problems:

$$FPR = \frac{FP}{N} = \frac{FP}{FP + TN}$$

$$TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

# Precision

- Measures the ratio of true positives (TP) to all positive predictions (TP + FP).
- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- Useful when false positives are costly
- In spam email detection, precision is valuable to ensure that legitimate emails are not classified as spam.

# Recall (Sensitivity) (True Positive Rate)

- Measures the ratio of true positives to all actual positives (TP + FN).
- Sensitivity measures the proportion of actual positives that the model correctly identifies as positive.
- It is also known as True Positive Rate (TPR).
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- Important when false negatives are costly.

# Specificity (True Negative Rate)

- Specificity measures the proportion of actual negatives that the model correctly identifies as negative.
- $\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$

# False Positive Rate (FPR)

- The False Positive Rate (FPR) is an important evaluation metric in binary classification that measures the proportion of actual negative instances that are incorrectly classified as positive by the model.

$$\text{FPR} = \frac{FP}{FP + TN}$$

# Trade-Off Between Sensitivity and Specificity

- **High Sensitivity, Low Specificity:** The model identifies most positive cases (high recall) but may incorrectly classify many negatives as positives, leading to a high false positive rate.
- **High Specificity, Low Sensitivity:** The model accurately identifies most negatives but misses many positive cases, resulting in a high false negative rate.

- **F1 Score:** A harmonic mean of precision and sensitivity, balancing both aspects when one metric alone is insufficient.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- True Positive (TP) = 3      False Negative (FN) = 1
- False Positive (FP) = 2      True Negative (TN) = 4

**3   1**

**2   4**



# QP Problem

- In the context of the e-commerce platform, assume that out of 1000 product recommendations, 750 successfully matched customer preferences, while the remaining 250 recommendations did not meet customer expectations. Furthermore, the system accurately identified 200 instances where the product suggestions were irrelevant. However, in 40 cases it incorrectly indicated product relevance when it was not, and in 10 instances, it failed to recommend products that aligned with customer preferences. Calculate the True Positives, True Negatives, False Positives, and False Negatives, as well as the accuracy, Precision, and recall for this application.

- True Positives (TP): Recommendations that successfully matched customer preferences.  
750 successful recommendations.  
TP = 750
- True Negatives (TN): Instances where the system correctly identified irrelevant recommendations.  
200 correctly identified irrelevant recommendations.  
TN = 200
- False Positives (FP): Cases where the system incorrectly indicated relevance (recommended irrelevant products).  
40 cases of incorrectly indicated relevance.  
FP = 40
- False Negatives (FN): Cases where the system failed to recommend relevant products that matched customer preferences.  
10 missed recommendations that aligned with preferences.  
FN = 10

# ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

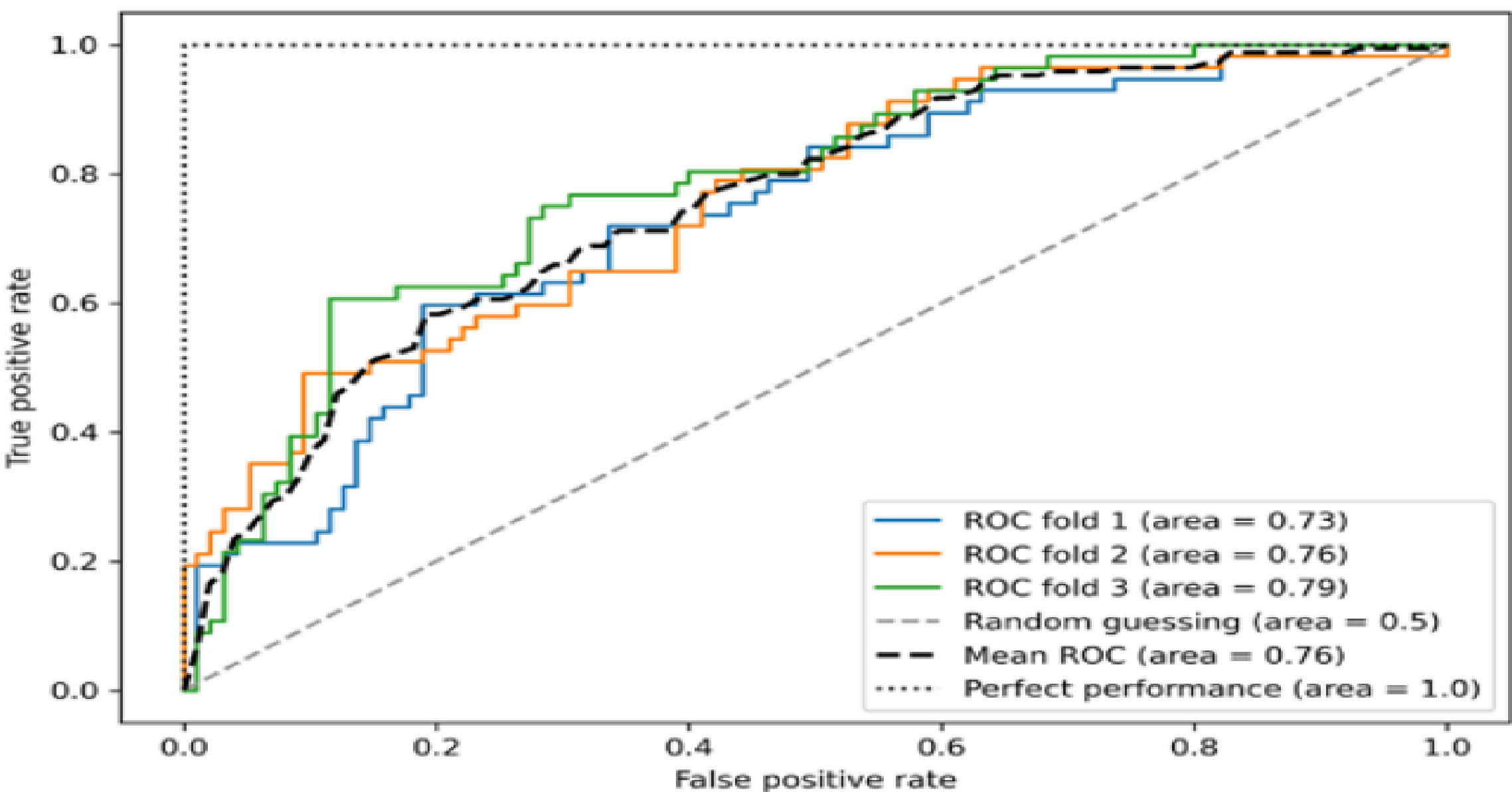
- ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings, and AUC measures the area under this curve.
- The AUC score represents the classifier's ability to discriminate between positive and negative classes.
- A perfect model has an AUC of 1, while a random model has an AUC of 0.5.
- Useful for evaluating model performance over various thresholds, particularly in binary classification tasks.

- Receiver operating characteristic (ROC) graphs are useful tools to select models for classification based on their performance with respect to the FPR and TPR, which are computed by shifting the decision threshold of the classifier.
- The diagonal of a ROC graph can be interpreted as random guessing, and classification models that fall below the diagonal are considered as worse than random guessing.
- A perfect classifier would fall into the top-left corner of the graph with a TPR of 1 and an FPR of 0.
- Based on the ROC curve, we can then compute the so-called ROC area under the curve (ROC AUC) to characterize the performance of a classification model.

•

**we will plot a ROC curve of a classifier that only uses two features from the Breast Cancer Wisconsin dataset to predict whether a tumor is benign or malignant.**

The resulting ROC curve indicates that there is a certain degree of variance between the different folds, and the average ROC AUC (0.76) falls between a perfect score (1.0) and random guessing (0.5)



# Matthews Correlation Coefficient (MCC)

- A correlation coefficient that takes into account all four categories in the confusion matrix, providing a balanced measure for binary classification.
- Useful for imbalanced datasets as it balances the effect of both classes.

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

# Cross-validated ROC curve for a binary classification task

**1. Dataset Generation:** `make_classification` generates a synthetic dataset for classification. It has 200 samples and 20 features. `train_test_split` splits the dataset into 70% training data and 30% test data. Selecting two features: `X_train2 = X_train[:, [4, 14]]` selects two features from the training set for simplicity in visualization.

**2. Pipeline Creation:** The `pipe_lr` is a machine learning pipeline

- Standard Scaler: Normalizes the features. `PCA(n_components=2)`  
Reduces the dimensionality of the data to 2 dimensions (for visualization).
- Logistic Regression: A logistic regression model is used for classification.

**3. Stratified K-Fold Cross-Validation `Stratified KFold(n_splits=3)`:** Performs 3-fold cross-validation while preserving the percentage of samples for each class. The `cv` object stores indices for training and testing in each fold.

**4. ROC Curve Calculation ROC Curve:** For each fold, roc\_curve is used to calculate the false positive rate (FPR) and true positive rate (TPR) for each threshold. This is then used to plot the ROC curve for each fold.

- **Interpolation:** The interp1d function is used to create smooth TPR curves across folds. Interpolation is the process of estimating values between two known values in a sequence. In the context of machine learning interpolation is often used to smooth curves, such as ROC (Receiver Operating Characteristic) curves, where to generate intermediate points between the actual data points to create a smooth line.

**5.AUC (Area Under the Curve):** The auc function calculates the AUC for each fold, which is a measure of the classifier's ability to distinguish between the positive and negative classes.



## 6. Plotting the Results:

The ROC curves for each fold are plotted with a label indicating the AUC value.

- A dashed line represents random guessing (AUC = 0.5).
- The mean ROC curve is plotted (average TPR over folds).
- A perfect performance line is plotted (AUC = 1.0).
- The final plot shows the ROC curve for each fold, the mean ROC curve, and the random and perfect performance lines

## 7. Output:

The ROC curves for each of the 3 folds in cross-validation.

- A mean ROC curve representing the average performance across folds.
- A line for random guessing and perfect performance for reference.
- X-axis: False Positive Rate (FPR).
- Y-axis: True Positive Rate (TPR).
- Legend: Displays AUC values for each fold and the mean ROC curve.

- A perfect classifier would fall into the top-left corner of the graph with a TPR of 1 and an FPR of 0.
- Based on the ROC curve, we can then compute the so-called ROC area under the curve (ROC AUC) to characterize the performance of a classification model.
- Similar to ROC curves, we can compute precision-recall curves for different probability thresholds of a classifier.
- A function for plotting those precision-recall curves is also implemented in scikit-learn.

# Precision-Recall Curve

- The Precision-Recall Curve is a graphical representation used to evaluate the performance of a classification model, particularly for imbalanced datasets.
- It illustrates the trade-off between precision and recall for different threshold values of a classifier.
- **High Precision & High Recall:** Indicates the model is performing well.
- **AUC (Area Under Curve):** The area under the PR curve (not to be confused with the ROC curve) is a single scalar value summarizing the performance.
- The closer the AUC is to 1, the better the model.

```
import numpy as np
from sklearn.metrics import precision_recall_curve, auc
import matplotlib.pyplot as plt

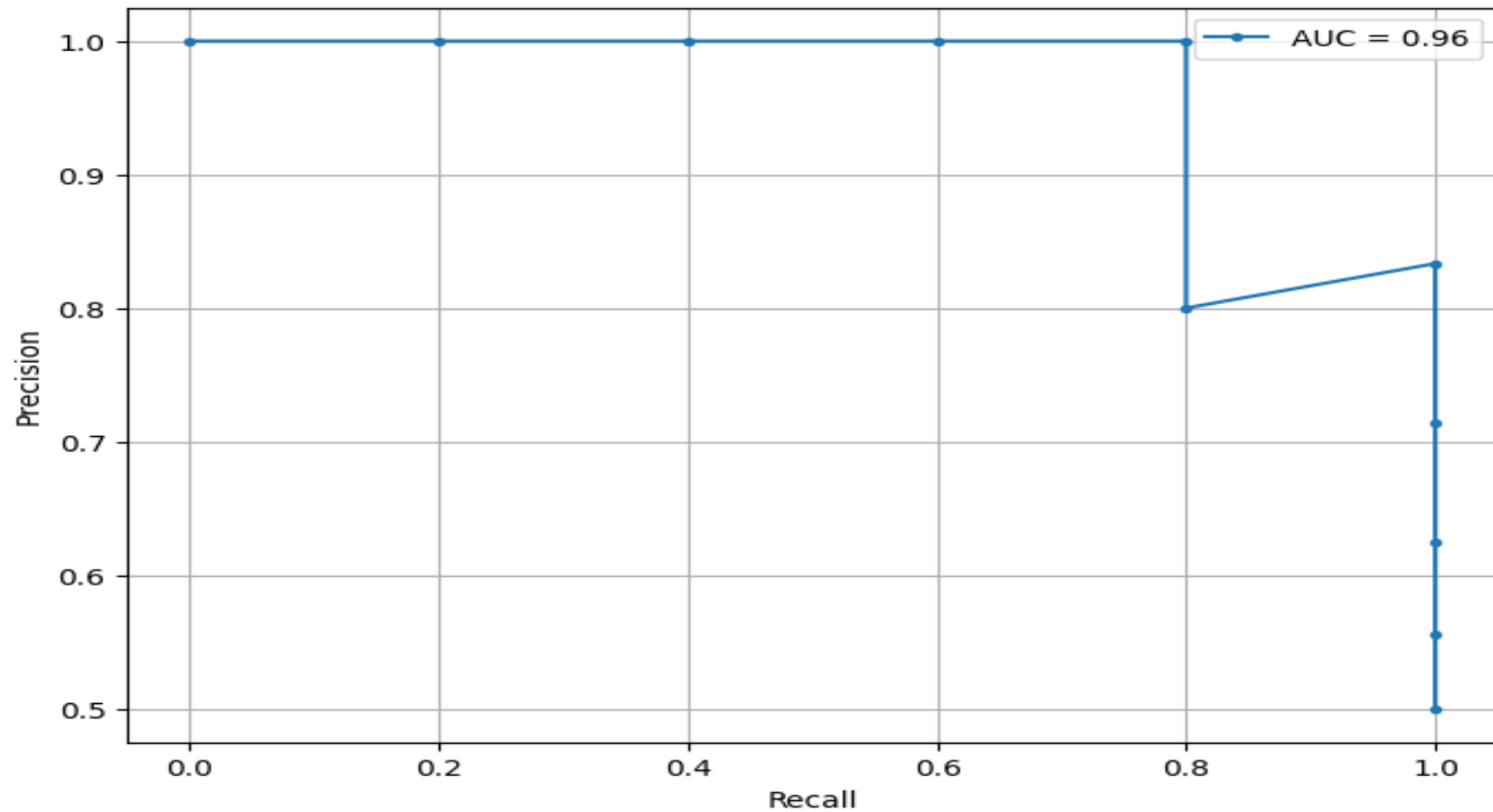
y_true = np.array([0, 0, 1, 1, 1, 0, 1, 0, 0, 1]) # True labels
y_scores = np.array([0.1, 0.4, 0.35, 0.8, 0.9, 0.2, 0.85, 0.05, 0.15, 0.7]) # Predicted probabilities

precision, recall, thresholds = precision_recall_curve(y_true, y_scores)

pr_auc = auc(recall, precision)

# Plot Precision-Recall Curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, marker='.', label=f'AUC = {pr_auc:.2f}')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid()
plt.show()
```

Precision-Recall Curve



# Regression metric for balanced and unbalanced data

- In regression, choosing the right metrics is crucial to evaluating model performance, especially for balanced or imbalanced data.
- For balanced data, all predictions are treated equally important, and metrics that assess overall error distribution work well.

**MAE, MSE, RMSE,  $R^2$  Score**

- For unbalanced data (with skewed distributions or outliers), metrics that are robust to outliers or emphasize relative errors are preferable.

**MAPE, SMAPE, Quantile Loss**

# Mean Absolute Error (MAE)

- Good for both balanced and imbalanced data.
- Measures the average magnitude of errors in predictions, providing a simple interpretation of the model's prediction accuracy.
- Measures the average magnitude of errors.
- It treats all errors equally and is robust to small variations.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

# Mean Squared Error (MSE)

- Typically used in balanced datasets sensitive to outliers, which may skew the metric for imbalanced data.
- Averages the square of errors, penalizing larger errors more heavily, which may make it unsuitable for imbalanced data if large errors are due to rare instances.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



# Root Mean Squared Error (RMSE)

- Similar to MSE, better suited for balanced data due to outlier sensitivity.
- The square root of MSE, keeping the error scale the same as the target variable, and emphasizing larger errors.
- Same as MSE but in the same units as the target variable.
- Often used for interpretability.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

# R-Squared (Coefficient of Determination)

- Suitable for both balanced and imbalanced data.
- Represents the proportion of variance in y that can be explained by X. Values range from 0 to 1, where higher values indicate a better fit.
- Measures the proportion of variance explained by the model.
- Useful for understanding goodness-of-fit.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

# Mean Absolute Percentage Error (MAPE)

- Works for both balanced and imbalanced data but depends on the absence of zero values in  $y_{\text{true}}$ .
- Measures error as a percentage, making it easy to interpret but sensitive to low target values.
- Provides a percentage error, making it scale-independent.
- Avoid using when  $y_i$  is close to zero (can result in extreme values).

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

# Symmetric Mean Absolute Percentage Error (SMAPE)

- SMAPE is a variation of MAPE that symmetrically penalizes over- and under-predictions, making it useful for imbalanced datasets.
- Normalizes the error based on the average of actual and predicted values.
- Suitable for skewed data(skewed data, the distribution of the target variable is asymmetric, meaning that it has a long tail on one side).

$$\text{SMAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2} \times 100$$

# Quantile Loss (Pinball Loss)

- Quantile Loss is a regression loss function used to predict specific quantiles of a target variable rather than the mean.
- Quantile loss, also known as pinball loss, calculates the error at a specified quantile, which is helpful for skewed distributions or for capturing variability in predictions.
- Useful when predicting specific percentiles (e.g., median or percentiles) instead of mean.

## **Quantile ( $\tau$ ):**

Specifies which quantile to predict.

$\tau=0.5$ : Median prediction.

$\tau=0.9$ : Upper quantile (90th percentile).

$\tau=0.1$ : Lower quantile (10th percentile).

## Error Penalty:

- When  $y_i > \hat{y}$ , the loss scales by  $\tau$
- .When  $y_i \leq \hat{y}$ , the loss scales by  $1-\tau$
- This asymmetry adjusts the importance of over- or under-predictions based on the quantile.

$$QL(y, \hat{y}; \tau) = \sum_{i=1}^n \begin{cases} \tau \cdot (y_i - \hat{y}_i) & \text{if } y_i > \hat{y}_i, \\ (1 - \tau) \cdot (\hat{y}_i - y_i) & \text{if } y_i \leq \hat{y}_i. \end{cases}$$

# Handling categorical variables

Handling categorical variables is essential in machine learning because most algorithms require numerical input.

There are several popular methods to handle categorical data, each suited to different types of models and problem requirements.

# 1.Label Encoding

Assigns a unique integer to each category. Suitable for: Ordinal categorical data (where categories have a meaningful order, e.g., 'low', 'medium', 'high').

```
from sklearn.preprocessing import LabelEncoder
```

```
data = ['low', 'medium', 'high', 'medium']
```

```
encoder = LabelEncoder()
```

```
encoded_data = encoder.fit_transform(data)
```

```
print(encoded_data)
```

Output: [1, 2, 0, 2]



## 2. One-Hot Encoding

- Creates binary columns for each category, with "1" representing presence and "0" representing absence.
- Suitable for Nominal categorical data (categories without an intrinsic order, e.g., colors or countries).

```
import pandas as pd
```

```
data = pd.DataFrame({'Color': ['Red', 'Blue', 'Green', 'Blue']})
```

```
one_hot_encoded_data = pd.get_dummies(data, columns=['Color'])
```

```
print(one_hot_encoded_data)
```

|   | Color_Blue | Color_Green | Color_Red |
|---|------------|-------------|-----------|
| 0 | False      | False       | True      |
| 1 | True       | False       | False     |
| 2 | False      | True        | False     |
| 3 | True       | False       | False     |

### 3. Binary Encoding

- Encodes each category as binary digits, reducing the number of columns compared to one-hot encoding, which can be helpful with high-cardinality features.
- Suitable for High-cardinality categorical data. High cardinality refers to categorical features that have a large number of unique values or categories.

```
import category_encoders as ce
```

```
data = pd.DataFrame({'Category': ['A', 'B', 'C', 'A', 'B', 'C', 'D']})
```

```
encoder = ce.BinaryEncoder(cols=['Category'])
```

```
binary_encoded_data = encoder.fit_transform(data)
```

```
print(binary_encoded_data)
```

|   | Category_0 | Category_1 | Category_2 |
|---|------------|------------|------------|
| 0 | 0          | 0          | 1          |
| 1 | 0          | 1          | 0          |
| 2 | 0          | 1          | 1          |
| 3 | 0          | 0          | 1          |
| 4 | 0          | 1          | 0          |
| 5 | 0          | 1          | 1          |
| 6 | 1          | 0          | 0          |

## 4. Frequency Encoding

- Replaces each category with its frequency (the count of each category).
- Suitable for Cases where the frequency of a category is meaningful, especially with high-cardinality features.

```
import pandas as pd
```

```
data = pd.DataFrame({'Fruit': ['Apple', 'Banana', 'Apple', 'Orange', 'Banana', 'Banana']})
```

```
data['Fruit_Frequency'] = data['Fruit'].map(data['Fruit'].value_counts())
```

```
print(data)
```

|   | Fruit  | Fruit_Frequency |
|---|--------|-----------------|
| 0 | Apple  | 2               |
| 1 | Banana | 3               |
| 2 | Apple  | 2               |
| 3 | Orange | 1               |
| 4 | Banana | 3               |
| 5 | Banana | 3               |

## 5.Mean/Target Encoding

Replaces each category with the mean of the target variable for that category.

This technique is common in categorical encoding for target prediction tasks.

```
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B', 'A'], 'Target': [1, 0, 1, 0, 1, 1]})
mean_encoded = data.groupby('Category')['Target'].mean()
data['Category_Target_Mean'] = data['Category'].map(mean_encoded)
print(data)
```

For Category 'A', the mean of Target is:  $1+1+1/3=1.0$

For Category 'B', the mean of Target is:  $0+1/2=0.5$

For Category 'C', the mean of Target is:  $0/1=0.0$

|   | Category | Target | Category_Target_Mean |
|---|----------|--------|----------------------|
| 0 | A        | 1      | 1.0                  |
| 1 | B        | 0      | 0.5                  |
| 2 | A        | 1      | 1.0                  |
| 3 | C        | 0      | 0.0                  |
| 4 | B        | 1      | 0.5                  |
| 5 | A        | 1      | 1.0                  |

## 6.Ordinal Encoding

Assigns an ordered integer to categories based on a predefined order. In this example 'low' will be encoded as 0, 'medium' as 1, and 'high' as 2.

```
from sklearn.preprocessing import OrdinalEncoder  
data = [['low'], ['medium'], ['high'], ['medium']]  
encoder = OrdinalEncoder(categories=[['low', 'medium', 'high']])  
ordinal_encoded = encoder.fit_transform(data)  
print(ordinal_encoded)
```

```
[[0.]  
 [1.]  
 [2.]  
 [1.] ]
```

## 7. Leave-One-Out Encoding

A variant of target encoding where the mean of the target variable for each category is calculated, excluding the current row to reduce overfitting. By excluding the current row, the encoding becomes unbiased and avoids using information that the model shouldn't have during training.

$$\text{Encoded Value} = \frac{\text{Sum of target values for same category(excluding current row)}}{\text{Count of rows for the same category(excluding current row)}}$$

```
import category_encoders as ce  
  
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B', 'A'], 'Target': [1, 0, 1, 0, 1, 1]})  
encoder = ce.LeaveOneOutEncoder(cols=['Category'])  
leave_one_out_encoded = encoder.fit_transform(data['Category'], data['Target'])  
print(leave_one_out_encoded)
```

## Example for B

- Identify rows where Category = 'B':
- Rows 1 and 4 have Category = 'B' with Target values [0, 1].
- Exclude the current row (Row 1): After excluding Row 1, the remaining Target value for Category = 'B' is [1] (from Row 4).
- Compute the mean using:

$$\text{Mean of the remaining Target values} = \frac{\text{Sum of remaining Target values}}{\text{count of remaining rows}}$$

Mean= 1/1 = 1 for B

|   | Category |
|---|----------|
| 0 | 1.000000 |
| 1 | 1.000000 |
| 2 | 1.000000 |
| 3 | 0.666667 |
| 4 | 0.000000 |
| 5 | 1.000000 |



# Regularization

- Regularization is one approach to tackling the problem of overfitting by adding additional information and thereby shrinking the parameter values of the model to induce a penalty against complexity.
- Regularization modifies the original loss function by adding a penalty term.
- Regularization is a technique in machine learning and statistics used to reduce overfitting, improve the generalization of models, and handle high-dimensional datasets.

- It works by adding a penalty term to the loss function to constrain the model's complexity, which discourages excessively large weights or coefficients.
- **Without regularization:** Models might overfit the training data, capturing noise and irrelevant patterns.
- **With regularization:** Models focus more on essential patterns and generalize better to unseen data.

# Why Regularization is Necessary?

- **Prevents Overfitting:** Overfitting occurs when a model performs well on the training data but poorly on test data. Regularization reduces this risk by penalizing large coefficients.
- **Handles Multicollinearity:** In datasets with highly correlated features, regularization helps by distributing weights effectively across features.
- **Improves Model Interpretability:** L1 regularization can eliminate irrelevant features, making models simpler and easier to interpret.
- **Enhances Robustness:** Regularized models are less sensitive to small variations in the training data.

# Generalized form of the regularized loss function

The generalized form of the regularized loss function

$$J(\theta) = \text{Loss}(\theta) + \lambda \cdot \text{Penalty}(\theta)$$

- $J(\theta)$ : Regularized loss function.
- $\text{Loss}(\theta)$  (e.g., Mean Squared Error)
- $\lambda$  Regularization strength (hyperparameter).
- $\text{Penalty}(\theta)$  Regularization term

# L1 Regularization (Lasso Regression)

Adds the absolute value of the coefficients as a penalty term to the loss function.

Loss Function with L1:  $J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n |\theta_i|$

- Suitable for datasets with many irrelevant features. This can result in feature selection.
- Encourages sparsity by shrinking some coefficients to exactly zero.
- Leads to sparse models where only the most important features remain.
- Useful for feature selection.
- Produces simpler models.
- Can be unstable when features are highly correlated.

# L2 Regularization (Ridge Regression)

- Adds the square of the coefficients as a penalty term to the loss function.

Loss Function with L2:  $J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n \theta_i^2$

- Shrinks coefficients, but not to zero.
- This leads to small, non-zero weights.
- Distributes the weight reduction more evenly among the features.
- Effective for datasets with multicollinearity.
- Prevents overfitting while retaining all features.
- Works well for problems with correlated features.
- Does not perform feature selection (all features are retained).

# Feature selection Techniques

Feature selection is an essential step in machine learning to reduce overfitting, improve model performance, and enhance interpretability.

**1. Variance Threshold Concept:** This method removes features with low variance.

- **Assumption:** Features with low variance contribute less to target prediction.
- Compute the variance for each feature. Drop features whose variance falls below a specified threshold (e.g., 0.01).
- Useful for datasets where some features are constant or nearly constant across samples.

Example: Removing a column where all values are nearly the same, such as [0.01, 0.02, 0.01].

## 2. Pearson Correlation (Heat Map)

- It measures the linear correlation between two features or between a feature and the target variable.
- Pearson correlation measures the strength and direction of the linear relationship between two continuous variables.
- It's one of the most widely used methods to quantify the degree to which two variables are related.
- Pearson correlation coefficient ( $r$ ), where :

$$r = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}}$$

$X_i, Y_i$  are the individual sample points of the variables  $X$  and  $Y$ .

$\bar{X}$  and  $\bar{Y}$  are the mean values of  $X$  and  $Y$ , respectively.

- Compute the correlation matrix of features.
- Visualize it using a heatmap.
- Drop one of the features in a highly correlated pair ( $|r| > \text{threshold}$  e.g., 0.80)
- Identifies redundant features.

Example: If two features  $X_1$  and  $X_2$  have  $r=0.95$  drop one of them.



# Interpretation of Pearson Correlation Coefficient

- $r=1$ : Perfect positive linear relationship. As  $X$  increases,  $Y$  increases proportionally.
- $r=-1$ : Perfect negative linear relationship. As  $X$  increases,  $Y$  decreases proportionally.
- $r=0$ : No linear relationship. Changes in  $X$  do not predict changes in  $Y$ .
- $0 < r < 1$ : Positive correlation. As  $X$  increases,  $Y$  tends to increase.
- $-1 < r < 0$ : Negative correlation. As  $X$  increases,  $Y$  tends to decrease.

Consider two variables,  $X$  (e.g., years of education) and  $Y$  (e.g., income). If there's a positive linear relationship, we would expect the Pearson correlation coefficient to be positive (closer to 1), suggesting that as education increases, income tends to increase.

# Correlation Matrix

- A correlation matrix is a table showing the Pearson correlation coefficients between many pairs of variables. It gives you a quick overview of how strongly variables are related to each other.

|   | A    | B    | C    |
|---|------|------|------|
| A | 1    | 0.85 | -0.4 |
| B | 0.85 | 1    | 0.1  |
| C | -0.4 | 0.1  | 1    |

# Heat Map of Pearson Correlation Matrix

- A heat map is a visual representation of the correlation matrix. It uses colors to represent the correlation values, making it easy to identify relationships between multiple variables at a glance.
- Calculate the Pearson correlation matrix for your dataset.
- Plot the correlation matrix using a heatmap.
- This can be done using libraries such as seaborn or matplotlib in Python.

## Color Gradient

- In the heatmap, the color gradient indicates the strength of the correlation.
- Hotter colors (yellow/green) indicate areas with higher click activity.
- Cooler colors (blue) represent areas with lower engagement.

**Annotations:** The numbers in each cell represent the exact Pearson correlation coefficient.

For instance, if two variables have a correlation of 0.85, the cell will display 0.85, and the color will indicate a strong positive relationship.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

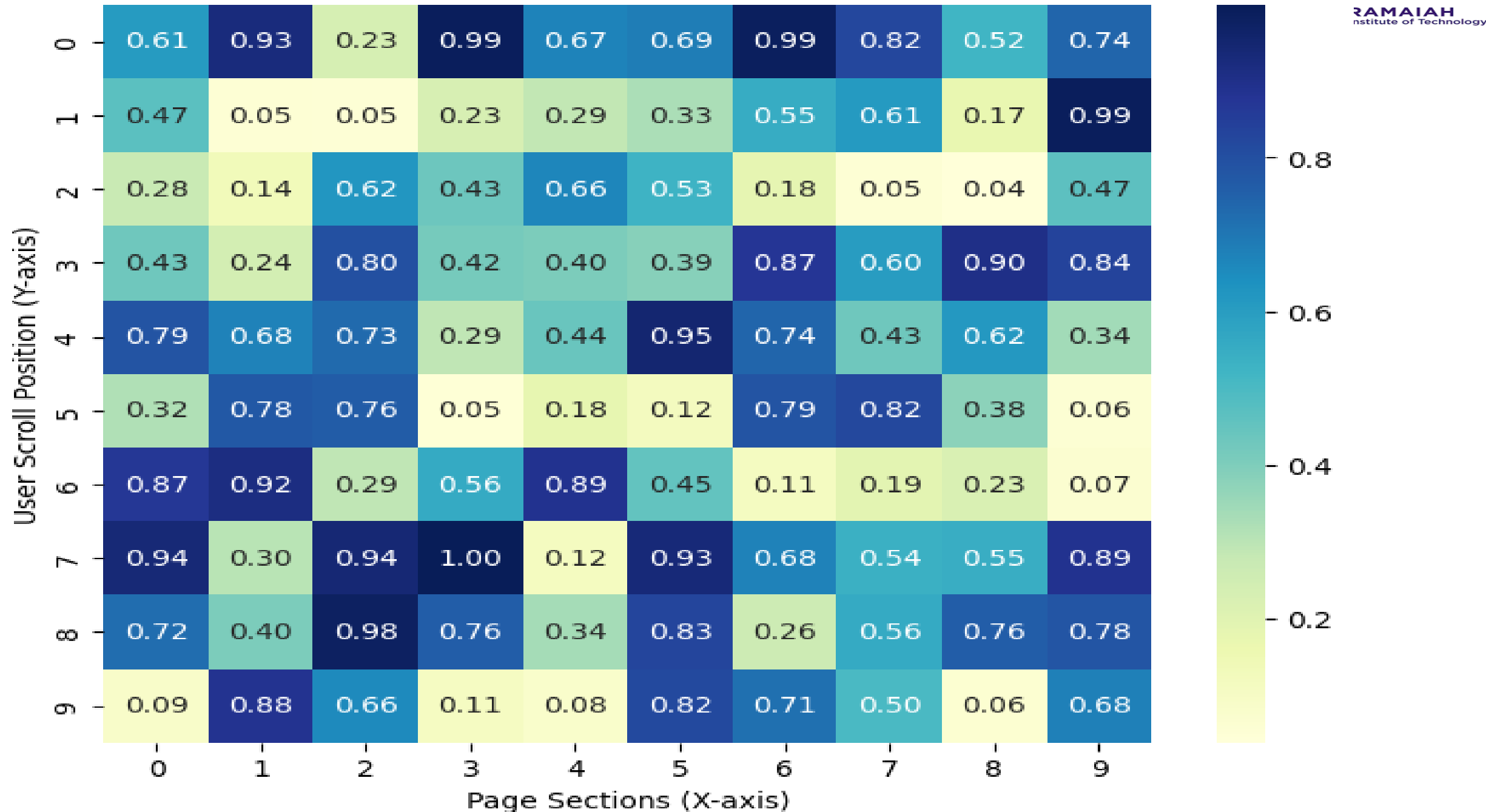
# Simulated data: A 10x10 grid representing clicks (higher values = more clicks)
data = np.random.rand(10, 10) # Randomized click data between 0 and 1

plt.figure(figsize=(8, 6))
sns.heatmap(data, annot=True, cmap='YlGnBu', fmt='.2f', cbar=True)

plt.title("Website Click Heatmap")
plt.xlabel("Page Sections (X-axis)")
plt.ylabel("User Scroll Position (Y-axis)")

plt.show()
```

# Website Click Heatmap



## 3. Greedy Feature Selection

- Incrementally adds or removes features to find a subset that maximizes a specified metric like accuracy, F1-score etc.
  - Forward Selection: Start with an empty feature set, then add features one at a time based on their contribution to performance.
  - Backward Elimination: Start with all features, then remove the least important feature iteratively.
  - Efficient for small to medium datasets, useful when you want a minimal feature subset.
1. Start with no features.
  2. Add feature  $X_1$  measure performance.
  3. Add  $X_2$ , and if the model improves, keep  $X_2$  otherwise, discard.

## 4. Recursive Feature Elimination (RFE)

Iteratively builds a model and removes the least important feature at each step.

1. Train a model (e.g., linear regression, SVM).
2. Rank features based on importance (e.g., weights in a linear model).
3. Remove the least important feature and retrain the model.
4. Repeat until the desired number of features is left.

Suitable for datasets where model interpretability is critical.

Use RFE to iteratively eliminate features contributing the least to classification accuracy.



## 5.Univariate Feature Selection

Select features based on their statistical significance in predicting the target.

Techniques:

- $\chi^2$  test for categorical features.
  - ANOVA F-test for continuous features.
  - Mutual information for both types.
1. Rank features based on their univariate test scores.
  2. Select the top  $k$  features or features exceeding a threshold score.

# Hypothesis

- A hypothesis is a proposed explanation or an educated guess about a phenomenon or a relationship between variables that can be tested through research and experimentation.
- It is a foundational element in the scientific method, guiding the direction of research and experiments.
- Hypotheses are usually framed in terms of relationships between variables, and once tested, they are either supported or refuted by empirical evidence.
- Hypothesis testing is essential in research for making informed decisions based on data.
- The goal is to gather evidence that either supports or contradicts the null hypothesis, guiding further exploration and understanding of the problem at hand.

# Types of Hypotheses

- **Null Hypothesis ( $H_0$ ):** The null hypothesis states that there is no effect, no relationship, or no difference between the variables being tested. It is a default assumption that researchers seek to test against.  
Example: "There is no difference in exam scores between students who study using textbooks and those who study using online courses."
- **Alternative Hypothesis ( $H_A$  or  $H_1$ ):** The alternative hypothesis suggests that there is an effect, a relationship, or a difference. It is what the researcher aims to prove.  
Example: "Students who study using online courses perform better than those who study using textbooks."

# Formulate a Hypothesis

- Step 1: Identify the research question. This could be something like, "Does smoking affect lung cancer risk?"
- Step 2: Determine the variables.
  - Independent Variable (IV): Smoking
  - Dependent Variable (DV): Lung cancer risk
- Step 3: Write the hypotheses.
  - Null Hypothesis ( $H_0$ ): Smoking does not affect the risk of lung cancer.
  - Alternative Hypothesis ( $H_A$ ): Smoking increases the risk of lung cancer.
- Step 4: Test the hypothesis using data collection, experiments, or statistical tests.

# Chi-Square Statistic

It quantifies the difference between the observed and expected frequencies.

**P-value:** Helps determine the statistical significance. If it's less than 0.05, we reject the null hypothesis.

The Chi-Square Test is used to determine if there is a significant association between two categorical variables.

It compares the observed frequencies with the expected frequencies under the null hypothesis.

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Consider a study where we want to determine if there is an association between gender (Male/Female) and preference for a type of music (Pop/Rock/Classical).

|        | Pop | Rock | Classical | Total |
|--------|-----|------|-----------|-------|
| Male   | 30  | 25   | 15        | 70    |
| Female | 40  | 35   | 25        | 100   |
| Total  | 70  | 60   | 40        | 170   |

- Contingency table observed frequency table summarizing the data.
- Null Hypothesis ( $H_0$ ): There is no association between gender and music preference.
- Alternative Hypothesis ( $H_A$ ): There is an association between gender and music preference.

$$E = (\text{Row Total}) \times (\text{Column Total}) / \text{Grand Total}$$

$$E_{\text{Male,Pop}} = 70 \times 70 / 170 = 28.82$$

Expected Table=

|       |       |       |
|-------|-------|-------|
| 28.82 | 24.71 | 16.47 |
| 41.18 | 35.29 | 23.53 |

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

$$\chi^2 = 0.31$$

Degrees of freedom = (Number of Rows - 1) × (Number of Columns - 1)  
 $df = (2 - 1) \times (3 - 1) = 2$

For small  $df$ , the distribution is highly skewed.

As  $df$  increases, the distribution becomes more symmetric and approaches a normal distribution.

- The Chi-Square Distribution Curve- A Chi-Square distribution is skewed to the right, with its shape depending on the degrees of freedom (df)
- CDF (Cumulative Distribution Function) calculates the area to the left of  $\chi^2=0.3108$
- Small right area (low p-value): The observed statistic is rare under  $H_0$ , so we reject  $H_0$
- Large right area (high p-value): The observed statistic is common under  $H_0$ , so we fail to reject  $H_0$ .
- The area under the Chi-Square distribution curve represents probabilities. It helps determine the likelihood of observing a Chi-Square statistic as extreme as (or more extreme than) the test statistic under the null hypothesis.